C Programming

NovoNex Delta



Introduction to Programming & Compilers

- **What is Programming?:** Writing precise instructions that a computer can execute to solve problems.
- **Why Learn Programming & C?:** C is the mother of modern languages. It builds deep intuition of computer memory, hardware control, and operating systems.
- **High-Level vs Machine Code:** Humans write human-readable source code (.c), but CPUs only understand binary 0s and 1s.
- **How Does a Compiler Work?:** A multi-stage translation pipeline:
- **1. Preprocessor:** Processes directives (#include, #define, macro expansions).
- **2. Compiler:** Translates preprocessed C code into Assembly code.
- **3. Assembler:** Converts Assembly code into machine object code (.o / .obj).
- **4. Linker:** Combines object files with system libraries into an executable (.exe / a.out).

💡 **Tip: GCC command: gcc -Wall file.c -o program**

✏️ Live Trace & Explanation Board

[Teacher's Note: Sketch the Compilation Pipeline Diagram (Source -> Preprocessor -> Compiler -> Assembler -> Linker -> Output) & RAM/CPU model here]

C Program Structure & Syntax Breakdown

- **`#include <stdio.h>`:** Header file for Standard Input/Output library (needed for printf).
- **`int main()`:** Entry-point function where computer begins executing instructions.
- **`printf(...)`:** Standard library function used to print formatted text to console.
- **`\n (Newline)`:** Escape character that moves the console cursor to the next line.
- **`return 0;`:** Tells the operating system that the program terminated successfully.
- **`Semicolon (;)`:** Every statement in C must end with a semicolon!

```c
// First C Program: Hello World
#include <stdio.h>


int main() {
    printf("Hello, World!\n");
    return 0;
}
```

💡 **Golden Rule: Execution always begins at main(), regardless of line position!**

✏️ Live Trace & Explanation Board

[Teacher's Note: Live trace cursor position with \n, demonstrate terminal compilation, and trace exit code 0 vs non-zero errors]

Variables, Data Types & Standard I/O

- **What is a Variable?:** A named container in RAM used to store and manipulate data values.
- **`int`:** Stores whole numbers (e.g. 42). Size: 4 bytes | Specifier: %d
- **long long:** Stores very large integers (up to ~9×10^18). Size: 8 bytes | Specifier: %lld
- **`char`:** Stores single ASCII character ('A', 'z'). Size: 1 byte | Specifier: %c
- **float:** Single-precision decimal (~6 digits). Size: 4 bytes | Specifier: %f
- **double:** Double-precision decimal (~15 digits). Size: 8 bytes | Specifier: %lf
- **`scanf(...) & & (Ampersand):`** Reads user input from keyboard. & passes the memory address!

```c
// Variable Declaration & I/O Example
int age; double gpa;
scanf("%d %lf", &age, &gpa);
printf("Age: %d, GPA: %.2lf\n", age, gpa);
```

⚠️ **Common Mistake: Forgetting '&' in scanf() causes segmentation faults!**

✏️ Live Trace & Explanation Board

[Teacher's Note: Draw RAM memory boxes with addresses (e.g. 0x7ffd..), bit sizes, and explain why &age gives the memory address to scanf]

Operators & Expression Evaluation

- **Arithmetic Operators:** +, -, *, / (quotient), % (remainder / modulo).
- **Integer Division:** 5 / 2 gives 2 (fraction truncated). 5.0 / 2 gives 2.5!
- **`Modulo Operator (%):`** Gives remainder: 7 % 3 = 1. (Works only on integers!).
- **Relational Operators:** ==, !=, <, >, <=, >= (Evaluates to 1 for TRUE, 0 for FALSE).
- **Logical Operators:** && (AND), || (OR), ! (NOT). Supports short-circuiting!
- **Assignment & Shorthand:** =, +=, -=, *=, /= (e.g., x += 5 is x = x + 5).
- **Increment / Decrement:** ++x (Pre: increment then use), x++ (Post: use then increment).

```c
// Expression Evaluation & Precedence
int a = 5, b = 2;
int res = a + b * 3;  // 5 + 6 = 11 (* before +)
int cond = (a > 3) && (b == 2); // 1 (TRUE)
```

💡 **Precedence Order: Parentheses () > (*, /, %) > (+, -) > Relational > Logical > Assignment**

✏️ Live Trace & Explanation Board

[Teacher's Note: Dry run complex expressions live: (e.g. a++ + ++a), integer truncation traps, and boolean truth tables]

Conditional Statements (if, else if, else)

- **Decision Making in C:** Controls execution flow based on boolean condition test (True/False).
- **In C: Non-Zero is TRUE:** 0 represents FALSE. Any non-zero value (-1, 1, 99) is TRUE!
- **`if` Statement:** Executes the enclosed block only if condition evaluates to TRUE.
- **`else if` Statement:** Tests alternative condition if preceding if was FALSE.
- **else Statement:** Default catch-all block; runs if all above conditions are FALSE.
- **Single-Path Execution:** As soon as ONE condition evaluates to TRUE, its block runs and the entire chain exits!

```c
            // Multi-Way Branching (Number Sign Check)
if (num > 0) {
    printf("Positive\n");
} else if (num < 0) {
    printf("Negative\n");
} else {
    printf("Zero\n");
}
```

⚠️ **Danger: if (x = 5) assigns 5 (always TRUE!). Always use == for comparison: if (x == 5)**

✏️ Live Trace & Explanation Board

[Teacher's Note: Draw Diamond Decision Flowchart here and test inputs: num = +10, num = -4, num = 0]

Nested Conditional Logic & Hierarchy

- **What is Nesting?:** Placing an if or if-else block inside another if or else block.
- **When to Use?:** When a secondary decision is dependent on a primary condition being satisfied.
- **Execution Hierarchy:** Inner condition is evaluated ONLY if the outer condition succeeds.
- **The Dangling Else Trap:** An else binds to the nearest preceding if without an else!
- **Best Practice:** Always enclose every block in curly braces {} even for 1 line!

```c
            // Nested Decision: Pass/Fail & Distinction
if (marks >= 40) {  // Outer Check: Passed?
    if (marks >= 80) {  // Inner Check: Honors?
        printf("Passed with Grade A+\n");
    } else {
        printf("Passed successfully\n");
    }
} else {
    printf("Failed. Please Retake\n");
}
```

💡 **Tip: Nested if-else can often be simplified using Logical Operators (&&, ||)!**

✏️ Live Trace & Explanation Board

[Teacher's Note: Trace complex problem live: Finding the largest of 3 numbers or Leap Year logic with nested tree]

Iterative Statements (Loops in C)

- **Why Use Loops?:** To repeat a block of code multiple times without rewriting statements.
- **3 Essential Loop Components::** Every loop requires:
- **1. Initialization:** Setting starting counter value (e.g. int i = 1).
- **2. Condition Test:** Boolean check evaluated before/after each cycle (i <= n).
- **3. Update / Step:** Modifying counter (i++, i += 2) to eventually break loop!
- **`for Loop:`** Best when number of iterations is known in advance.
- **`while Loop:`** Entry-controlled loop; checks condition BEFORE running body.
- **`do-while Loop:`** Exit-controlled loop; GUARANTEED to execute at least ONCE!

```c
// Comparison of Loop Syntaxes (Print 1 to 5)
// 1. for loop
for (int i = 1; i <= 5; i++) { printf("%d ", i); }
// 2. while loop
int i = 1; while (i <= 5) { printf("%d ", i); i++; }
// 3. do-while loop (runs at least once)
int i = 1; do { printf("%d ", i); i++; } while (i <= 5);
```

⚠️ **Infinite Loop Danger: If the update step (i++) is omitted, condition stays true forever!**

✏️ Live Trace & Explanation Board

[Teacher's Note: Draw Iteration Trace Table live: Step | i value | Condition (i<=5) | Output | New i]

Nested Loops & Multi-Dimensional Traversal

- **Concept of Nested Loops:** A loop placed inside the body of another loop.
- **How Nested Loops Run::** For every SINGLE iteration of the outer loop, the inner loop completes its ENTIRE iteration cycle!
- **Total Iterations Formula:** Total executions = (Outer Count) × (Inner Count).
- **Time Complexity:** Two loops running N times results in O(N × N) = O(N^2).
- **Primary Applications:** Printing 2D star patterns, coordinate grids, and traversing 2D matrices/tables.

```
            // Pattern Example: 3 Rows x 4 Stars Grid
for (int r = 1; r <= 3; r++) {      // Outer Loop (Rows)
    for (int c = 1; c <= 4; c++) {  // Inner Loop (Cols)
        printf("* ");
    }
    printf("\n");              // Move to next row
}
```

💡 **Pattern Rule: Outer loop always controls Rows (r); Inner loop controls Columns (c)!**

✏️ Live Trace & Explanation Board

[Teacher's Note: Trace live row-column matrix, variable values (r, c), and draw triangular pyramid patterns: (c <= r)]

1D Arrays: Memory Layout & Indexing

- **What is an Array?:** A contiguous collection of elements of the same data type stored in consecutive RAM memory cells.
- **Fixed Size:** Array capacity is fixed at declaration: int arr[5];
- **0-Based Indexing:** Indices always start at 0 and end at (size - 1):
- **Valid Indices for size 5:** arr[0], arr[1], arr[2], arr[3], arr[4].
- **Contiguous Memory Address:** Each integer occupies 4 bytes. Address jumps by 4:
- **Formula:** Address(arr[i]) = Base_Address + (i × sizeof(type)).
- **Array Traversal:** Use a standard for loop from i = 0 to i < size.

```c
// 1D Array Declaration, Input & Output
int arr[5] = {10, 20, 30, 40, 50};
for (int i = 0; i < 5; i++) {
    printf("arr[%d] = %d\n", i, arr[i]);
}
```

⚠️ **Out of Bounds: Accessing arr[5] in a size-5 array leads to undefined behavior / garbage memory!**

✏️ Live Trace & Explanation Board

[Teacher's Note: Draw continuous RAM memory boxes, show index 0..4 vs memory addresses (1000, 1004, 1008..), and pointer decay]

2D Arrays (Matrices): Grid & Memory Mapping

- **What is a 2D Array?:** An array of arrays, conceptually organized as a table/grid of Rows and Columns.
- **Declaration Syntax:** dataType name[ROWS][COLUMNS]; (e.g. int matrix[3][4];).
- **Element Access:** matrix[r][c] accesses the item at Row r and Column c.
- **Nested Loop Traversal:** Outer loop iterates rows (i); Inner loop iterates cols (j).
- **Row-Major Memory Order:** In C, 2D arrays are stored linearly in RAM row-by-row (Row 0 elements, then Row 1 elements, etc.).

```c
// 2D Array: Reading & Printing a Matrix
int mat[2][3] = {{1, 2, 3}, {4, 5, 6}};
for (int i = 0; i < 2; i++) {      // Rows
    for (int j = 0; j < 3; j++) {  // Columns
        printf("%d ", mat[i][j]);
    }
    printf("\n");
}
```

💡 **Linear Offset Formula: Address(mat[i][j]) = Base + (i × COLS + j) × sizeof(type)**

✏️ Live Trace & Explanation Board

[Teacher's Note: Draw 2x3 Matrix Grid [row][col] and show the linear 1D memory unrolling (Row-Major Storage) in RAM]

Thank You

Happy Coding with C & NovoNex Delta!

